JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class AtomicReferenceArray<E>

java.lang.Object
  java.util.concurrent.atomic.AtomicReferenceArray<E>

Type Parameters:

E - The base class of elements held in this array

All Implemented Interfaces:

Serializable

```
public class AtomicReferenceArray<E>
extends Object
implements Serializable
```

An array of object references in which elements may be updated atomically. See the VarHandle specification for descriptions of the properties of atomic accesses.

Since:

1.5

See Also:

Serialized Form

Constructor Summary

Constructors

Constructor	Description
AtomicReferenceArray(int length)	Creates a new AtomicReferenceArray of the given length, with all elements initially null.
AtomicReferenceArray(E[] array)	Creates a new AtomicReferenceArray with the same length as, and all elements copied from, the given array.

Method Summary

All Methods | **Instance Methods**

Modifier and Type	Method	Description
final E		

	accumulateAndGet(int i, E x, BinaryOperator<E> accumu	Atomically updates (with memory effects as specified by VarHandle.compareAndSet(Object...)) the element at index i with the results of applying the given function to the current and given values, returning the updated value.
final E	**compareAndExchange** (int i, E expectedValue, E newValue)	Atomically sets the element at index i to newValue if the element's current value, referred to as the *witness value*, == expectedValue, with memory effects as specified by VarHandle.compareAndExchange(Object...).
final E	**compareAndExchangeAcquir** (int i, E expectedValue, E newValue)	Atomically sets the element at index i to newValue if the element's current value, referred to as the *witness value*, == expectedValue, with memory effects as specified by VarHandle.compareAndExchangeAcquire(Obje
final E	**compareAndExchangeReleas** (int i, E expectedValue, E newValue)	Atomically sets the element at index i to newValue if the element's current value, referred to as the *witness value*, == expectedValue, with memory effects as specified by VarHandle.compareAndExchangeRelease(Obje
final boolean	**compareAndSet**(int i, E expectedValue, E newValue)	Atomically sets the element at index i to newValue if the element's

		current value == expectedValue, with memory effects as specified by VarHandle.compareAndSet(Object...).
final E	get(int i)	Returns the current value of the element at index i, with memory effects as specified by VarHandle.getVolatile(Object...).
final E	getAcquire(int i)	Returns the current value of the element at index i, with memory effects as specified by VarHandle.getAcquire(Object...).
final E	getAndAccumulate(int i, E x, BinaryOperator<E> accumu	Atomically updates (with memory effects as specified by VarHandle.compareAndSet(Object...)) the element at index i with the results of applying the given function to the current and given values, returning the previous value.
final E	getAndSet(int i, E newValue)	Atomically sets the element at index i to newValue and returns the old value, with memory effects as specified by VarHandle.getAndSet(Object...).
final E	getAndUpdate(int i, UnaryOperator<E> updateF	Atomically updates (with memory effects as specified by VarHandle.compareAndSet(Object...))

		the element at index i with the results of applying the given function, returning the previous value.
final E	**getOpaque**(int i)	Returns the current value of the element at index i, with memory effects as specified by VarHandle.getOpaque(Object...).
final E	**getPlain**(int i)	Returns the current value of the element at index i, with memory semantics of reading as if the variable was declared non-volatile.
final void	**lazySet**(int i, E newValue)	Sets the element at index i to newValue, with memory effects as specified by VarHandle.setRelease(Object...).
final int	**length**()	Returns the length of the array.
final void	**set**(int i, E newValue)	Sets the element at index i to newValue, with memory effects as specified by VarHandle.setVolatile(Object...).
final void	**setOpaque**(int i, E newValue)	Sets the element at index i to newValue, with memory effects as specified by VarHandle.setOpaque(Object...).
final void	**setPlain**(int i, E newValue)	Sets the element at index i to newValue, with

		memory semantics of setting as if the variable was declared non-`volatile` and non-`final`.
`final void`	**`setRelease`**`(int i, E newValue)`	Sets the element at index `i` to `newValue`, with memory effects as specified by `VarHandle.setRelease(Object...)`.
`String`	**`toString`**`()`	Returns the String representation of the current values of array.
`final E`	**`updateAndGet`**`(int i, UnaryOperator<E> updateF`	Atomically updates (with memory effects as specified by `VarHandle.compareAndSet(Object...)`) the element at index `i` with the results of applying the given function, returning the updated value.
`final boolean`	**`weakCompareAndSet`**`(int i, E expectedValue, E newValue)`	**Deprecated.** This method has plain memory effects but the method name implies volatile memory effects (see methods such as `compareAndExchange(int, E, E)` and `compareAndSet(int, E, E)`).
`final boolean`	**`weakCompareAndSetAcquire`**`(int i, E expectedValue, E newValue)`	Possibly atomically sets the element at index `i` to `newValue` if the element's current value == `expectedValue`, with memory effects as

		specified by VarHandle.weakCompareAndSetAcquire(Objec
final boolean	**weakCompareAndSetPlain**(int i, E expectedValue, E newValue)	Possibly atomically sets the element at index i to newValue if the element's current value == expectedValue, with memory effects as specified by VarHandle.weakCompareAndSetPlain(Object.
final boolean	**weakCompareAndSetRelease**(int i, E expectedValue, E newValue)	Possibly atomically sets the element at index i to newValue if the element's current value == expectedValue, with memory effects as specified by VarHandle.weakCompareAndSetRelease(Objec
final boolean	**weakCompareAndSetVolatil**(int i, E expectedValue, E newValue)	Possibly atomically sets the element at index i to newValue if the element's current value == expectedValue, with memory effects as specified by VarHandle.weakCompareAndSet(Object...).

Methods declared in class Object

`clone`, `equals`, `finalize`, `getClass`, `hashCode`, `notify`, `notifyAll`, `wait`, `wait`, `wait`

Constructor Details

AtomicReferenceArray

```
public AtomicReferenceArray(int length)
```

Creates a new AtomicReferenceArray of the given length, with all elements initially null.

Parameters:

`length` - the length of the array

AtomicReferenceArray

```
public AtomicReferenceArray(E[] array)
```

Creates a new AtomicReferenceArray with the same length as, and all elements copied from, the given array.

Parameters:

`array` - the array to copy elements from

Throws:

`NullPointerException` - if array is null

Method Details

length

```
public final int length()
```

Returns the length of the array.

Returns:

the length of the array

get

```
public final E get(int i)
```

Returns the current value of the element at index `i`, with memory effects as specified by `VarHandle.getVolatile(Object...)`.

Parameters:

i - the index

Returns:

the current value

set

```
public final void set(int i,
                      E newValue)
```

Sets the element at index i to newValue, with memory effects as specified by VarHandle.setVolatile(Object...).

Parameters:

i - the index

newValue - the new value

lazySet

```
public final void lazySet(int i,
                          E newValue)
```

Sets the element at index i to newValue, with memory effects as specified by VarHandle.setRelease(Object...).

Parameters:

i - the index

newValue - the new value

Since:

1.6

getAndSet

```
public final E getAndSet(int i,
                         E newValue)
```

Atomically sets the element at index i to newValue and returns the old value, with memory effects as specified by VarHandle.getAndSet(Object...).

Parameters:

i - the index

newValue - the new value

Returns:

the previous value

compareAndSet

```
public final boolean compareAndSet(int i,
                                   E expectedValue,
                                   E newValue)
```

Atomically sets the element at index `i` to `newValue` if the element's current value == `expectedValue`, with memory effects as specified by `VarHandle.compareAndSet(Object...)`.

Parameters:

`i` - the index

`expectedValue` - the expected value

`newValue` - the new value

Returns:

`true` if successful. False return indicates that the actual value was not equal to the expected value.

weakCompareAndSet

```
@Deprecated(since="9")
public final boolean weakCompareAndSet(int i,
                                       E expectedValue,
                                       E newValue)
```

Deprecated.
This method has plain memory effects but the method name implies volatile memory effects (see methods such as `compareAndExchange(int, E, E)` and `compareAndSet(int, E, E)`). To avoid confusion over plain or volatile memory effects it is recommended that the method `weakCompareAndSetPlain(int, E, E)` be used instead.

Possibly atomically sets the element at index `i` to `newValue` if the element's current value == `expectedValue`, with memory effects as specified by `VarHandle.weakCompareAndSetPlain(Object...)`.

Parameters:

`i` - the index

`expectedValue` - the expected value

`newValue` - the new value

Returns:

`true` if successful

See Also:

`weakCompareAndSetPlain(int, E, E)`

weakCompareAndSetPlain

```
public final boolean weakCompareAndSetPlain(int i,
                                            E expectedValue,
                                            E newValue)
```

Possibly atomically sets the element at index `i` to `newValue` if the element's current value == `expectedValue`, with memory effects as specified by `VarHandle.weakCompareAndSetPlain(Object...)`.

Parameters:

`i` - the index

`expectedValue` - the expected value

`newValue` - the new value

Returns:

`true` if successful

Since:

9

getAndUpdate

```
public final E getAndUpdate(int i,
                            UnaryOperator<E> updateFunction)
```

Atomically updates (with memory effects as specified by `VarHandle.compareAndSet(Object...)`) the element at index `i` with the results of applying the given function, returning the previous value. The function should be side-effect-free, since it may be re-applied when attempted updates fail due to contention among threads.

Parameters:

`i` - the index

`updateFunction` - a side-effect-free function

Returns:

the previous value

Since:

1.8

updateAndGet

```
public final E updateAndGet(int i,
                            UnaryOperator<E> updateFunction)
```

Atomically updates (with memory effects as specified by `VarHandle.compareAndSet(Object...)`) the element at index `i` with the results of applying the given function, returning the updated value. The function should be side-effect-free, since it may be re-applied when attempted updates fail due to contention among threads.

Parameters:

`i` - the index

`updateFunction` - a side-effect-free function

Returns:

the updated value

Since:

1.8

getAndAccumulate

```
public final E getAndAccumulate
(int i,
 E x,
 BinaryOperator<E> accumulatorFunction)
```

Atomically updates (with memory effects as specified by `VarHandle.compareAndSet(Object...)`) the element at index `i` with the results of applying the given function to the current and given values, returning the previous value. The function should be side-effect-free, since it may be re-applied when attempted updates fail due to contention among threads.
The function is applied with the current value of the element at index `i` as its first argument, and the given update as the second argument.

Parameters:

`i` - the index

`x` - the update value

`accumulatorFunction` - a side-effect-free function of two arguments

Returns:

the previous value

Since:

1.8

accumulateAndGet

```
public final E accumulateAndGet
(int i,
 E x,
 BinaryOperator<E> accumulatorFunction)
```

Atomically updates (with memory effects as specified by `VarHandle.compareAndSet(Object...)`) the element at index `i` with the results of applying the given function to the current and given values, returning the updated value. The function should be side-effect-free, since it may be re-applied when attempted updates fail due to contention among threads.
The function is applied with the current value of the element at index `i` as its first argument, and the given update as the second argument.

Parameters:

`i` - the index

`x` - the update value

`accumulatorFunction` - a side-effect-free function of two arguments

Returns:

the updated value

Since:

1.8

toString

```
public String toString()
```

Returns the String representation of the current values of array.

Overrides:

`toString` in class `Object`

Returns:

the String representation of the current values of array

getPlain

```
public final E getPlain(int i)
```

Returns the current value of the element at index `i`, with memory semantics of reading as if the variable was declared non-`volatile`.

Parameters:

`i` - the index

Returns:

the value

Since:

9

setPlain

```
public final void setPlain(int i,
                           E newValue)
```

Sets the element at index `i` to `newValue`, with memory semantics of setting as if the variable was declared non-`volatile` and non-`final`.

Parameters:

`i` - the index

`newValue` - the new value

Since:

9

getOpaque

```
public final E getOpaque(int i)
```

Returns the current value of the element at index `i`, with memory effects as specified by `VarHandle.getOpaque(Object...)`.

Parameters:

`i` - the index

Returns:

the value

Since:

9

setOpaque

```
public final void setOpaque(int i,
                            E newValue)
```

Sets the element at index `i` to `newValue`, with memory effects as specified by `VarHandle.setOpaque(Object...)`.

Parameters:

`i` - the index

`newValue` - the new value

Since:

9

getAcquire

```
public final E getAcquire(int i)
```

Returns the current value of the element at index `i`, with memory effects as specified by `VarHandle.getAcquire(Object...)`.

Parameters:

`i` - the index

Returns:

the value

Since:

9

setRelease

```
public final void setRelease(int i,
                             E newValue)
```

Sets the element at index `i` to `newValue`, with memory effects as specified by `VarHandle.setRelease(Object...)`.

Parameters:

`i` - the index

`newValue` - the new value

Since:

9

compareAndExchange

```
public final E compareAndExchange(int i,
                                  E expectedValue,
                                  E newValue)
```

Atomically sets the element at index `i` to `newValue` if the element's current value, referred to as the *witness value*, `==` `expectedValue`, with memory effects as specified by `VarHandle.compareAndExchange(Object...)`.

Parameters:

`i` - the index

`expectedValue` - the expected value

`newValue` - the new value

Returns:

the *witness value*, which will be the same as the expected value if successful

Since:

9

compareAndExchangeAcquire

```
public final E compareAndExchangeAcquire(int i,
                                         E expectedValue,
                                         E newValue)
```

Atomically sets the element at index `i` to `newValue` if the element's current value, referred to as the *witness value*, `==` `expectedValue`, with memory effects as specified by `VarHandle.compareAndExchangeAcquire(Object...)`.

Parameters:

`i` - the index

`expectedValue` - the expected value

`newValue` - the new value

Returns:

the *witness value*, which will be the same as the expected value if successful

Since:

9

compareAndExchangeRelease

```
public final E compareAndExchangeRelease(int i,
                                         E expectedValue,
                                         E newValue)
```

Atomically sets the element at index `i` to `newValue` if the element's current value, referred to as the *witness value*, `==` `expectedValue`, with memory effects as specified by `VarHandle.compareAndExchangeRelease(Object...)`.

Parameters:

`i` - the index

`expectedValue` - the expected value

`newValue` - the new value

Returns:

the *witness value*, which will be the same as the expected value if successful

Since:

9

weakCompareAndSetVolatile

```
public final boolean weakCompareAndSetVolatile(int i,
                                               E expectedValue,
                                               E newValue)
```

Possibly atomically sets the element at index `i` to `newValue` if the element's current value `==` `expectedValue`, with memory effects as specified by `VarHandle.weakCompareAndSet(Object...)`.

Parameters:

`i` - the index

`expectedValue` - the expected value

`newValue` - the new value

Returns:

`true` if successful

Since:

9

weakCompareAndSetAcquire

```
public final boolean weakCompareAndSetAcquire(int i,
                                              E expectedValue,
                                              E newValue)
```

Possibly atomically sets the element at index `i` to `newValue` if the element's current value `==` `expectedValue`, with memory effects as specified by `VarHandle.weakCompareAndSetAcquire(Object...)`.

Parameters:

`i` - the index

`expectedValue` - the expected value

`newValue` - the new value

Returns:

`true` if successful

Since:

9

weakCompareAndSetRelease

```
public final boolean weakCompareAndSetRelease(int i,
                                              E expectedValue,
                                              E newValue)
```

Possibly atomically sets the element at index `i` to `newValue` if the element's current value == `expectedValue`, with memory effects as specified by `VarHandle.weakCompareAndSetRelease(Object...)`.

Parameters:

`i` - the index

`expectedValue` - the expected value

`newValue` - the new value

Returns:

`true` if successful

Since:

9

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.